

## **Software Construction and Development Project Report**



**SOFTWARE CONSTRUCTION AND DEVELOPMENT**

**PROJECT SPRING 2025**

**DONE BY 22I-1529 (Amna Noor), 22I-2415 (Kashaf Fatima)**

**Submitted To : Sir Junaid Ali**

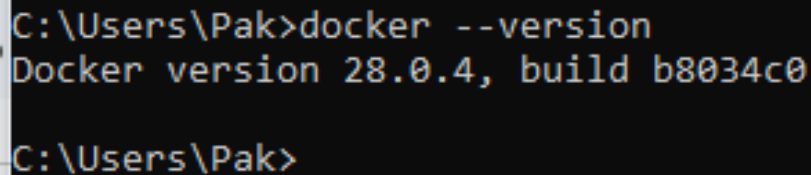
## Environment Setup

System & Tool Versions:

- Operating System: Windows 10 Home
- Docker Version: Docker version 28.0.4
- Minikube Version: v1.33.0
- kubectl Version: v1.29.0
- Package Manager: Chocolatey

### Install Minikube and kubectl:

- Docker Version:
- Command: `docker --version`



```
C:\Users\Pak>docker --version
Docker version 28.0.4, build b8034c0
C:\Users\Pak>
```

### Install Minikube:

For installing minikube and kubectl we are going to use chocolatey package manager. Here are the steps to install chocolatey package manager:

### Installing Chocolatey:

Installs Chocolatey, a package manager for Windows, by temporarily bypassing the PowerShell execution policy, ensuring secure web

communication, and downloading and executing the official Chocolatey installation script.

**Set-ExecutionPolicy Bypass -Scope Process -Force; `**

**[System.Net.ServicePointManager]::SecurityProtocol = `**

**[System.Net.ServicePointManager]::SecurityProtocol -bor 3072; `**

**ieX ((New-Object**

**System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps1'))**

```
PS C:\WINDOWS\system32> Set-ExecutionPolicy Bypass -Scope Process -Force; `
>> [System.Net.ServicePointManager]::SecurityProtocol = `
>> [System.Net.ServicePointManager]::SecurityProtocol -bor 3072; `
>> ieX ((New-Object System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps1'))
>>
Forcing web requests to allow TLS v1.2 (Required for requests to Chocolatey.org)
Getting latest version of the Chocolatey package for download.
Not using proxy.
Getting Chocolatey from https://community.chocolatey.org/api/v2/package/chocolatey/2.4.3.
Downloading https://community.chocolatey.org/api/v2/package/chocolatey/2.4.3 to C:\Users\Pak\AppData\Local\Temp\chocolatey\chocoInstall\chocolatey.zip
Not using proxy.
Extracting C:\Users\Pak\AppData\Local\Temp\chocolatey\chocoInstall\chocolatey.zip to C:\Users\Pak\AppData\Local\Temp\chocolatey\chocoInstall
Installing Chocolatey on the local machine
Creating ChocolateyInstall as an environment variable (targeting 'Machine')
  Setting ChocolateyInstall to 'C:\ProgramData\chocolatey'
WARNING: It's very likely you will need to close and reopen your shell
  before you can use choco.
Restricting write permissions to Administrators
We are setting up the Chocolatey package repository.
The packages themselves go to 'C:\ProgramData\chocolatey\lib'
  (i.e. C:\ProgramData\chocolatey\lib\yourPackageName).
A shim file for the command line goes to 'C:\ProgramData\chocolatey\bin'
  and points to an executable in 'C:\ProgramData\chocolatey\lib\yourPackageName'.
Creating Chocolatey CLI folders if they do not already exist.
```

```
Ensuring chocolatey.nupkg is in the lib folder
PS C:\WINDOWS\system32> choco --version
2.4.3
PS C:\WINDOWS\system32> choco install minikube
Chocolatey v2.4.3
Installing the following packages:
minikube
By installing, you accept licenses for the packages.
```

```

Downloading package from source 'https://community.chocolatey.org/api/v2/'
Progress: Downloading Minikube 1.35.0... 100%

Minikube v1.35.0 [Approved]
Minikube package files install completed. Performing other installation steps.
ShimGen has successfully created a shim for minikube.exe
The install of Minikube was successful.
  Deployed to 'C:\ProgramData\chocolatey\lib\Minikube'

Chocolatey installed 2/2 packages.
  See the log for details (C:\ProgramData\chocolatey\logs\chocolatey.log).
PS C:\WINDOWS\system32>

```

- Install **kubectl** (using chocolatey)

### choco install kubernetes-cli

```

See the log for details (C:\ProgramData\chocolatey\logs\chocolatey.log).
PS C:\WINDOWS\system32> choco install kubernetes-cli
Chocolatey v2.4.3
Installing the following packages:
kubernetes-cli
By installing, you accept licenses for the packages.
kubernetes-cli v1.33.0 already installed.
  Use --force to reinstall, specify a version to install, or try upgrade.

Chocolatey installed 0/1 packages.
  See the log for details (C:\ProgramData\chocolatey\logs\chocolatey.log).

Warnings:
 - kubernetes-cli - kubernetes-cli v1.33.0 already installed.
  Use --force to reinstall, specify a version to install, or try upgrade.
PS C:\WINDOWS\system32>

```

This command uses Chocolatey to install the `kubectl` CLI tool, which is the command-line interface for interacting with Kubernetes clusters.

- **Verifying Installation:**

`choco install minikube` – Installs Minikube, a tool to run a local Kubernetes cluster, using Chocolatey.

`minikube version` – Displays the installed version of Minikube to verify successful installation.

`kubectl version --client` – Shows the client version of kubectl, confirming that the CLI is installed correctly.

```
Use --force to reinstall, specify a version to install, or try upgrade.  
PS C:\WINDOWS\system32> minikube version  
minikube version: v1.35.0  
commit: dd5d320e41b5451cdf3c01891bc4e13d189586ed-dirty  
PS C:\WINDOWS\system32> kubectl version --client  
Client Version: v1.33.0  
Kustomize Version: v5.6.0  
PS C:\WINDOWS\system32>
```

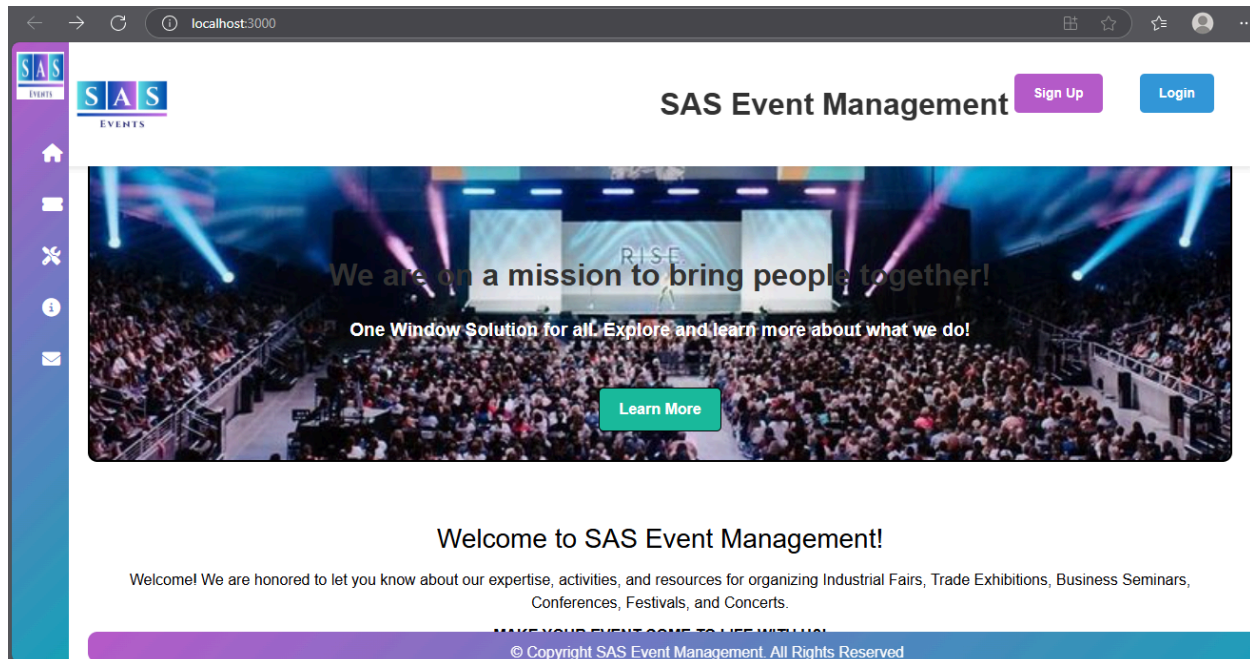
**choco install minikube**

**minikube version**

**kubectl version --client**

## 1. Develop a Web App

Utilizing the existing project of the previous semester which was an event management web app having multiple pages along with mongodb.



## 2. Containerized the Application

### Docker File:

```
# ----- FRONTEND BUILD STAGE -----
FROM node:18 as frontend

WORKDIR /app/frontend

COPY frontend/package*.json ./
RUN npm install
COPY frontend/ ./
RUN npm run build

# ----- BACKEND + FINAL IMAGE -----
FROM node:18

WORKDIR /app
```

```
# Install backend dependencies
COPY backend/package*.json ./backend/
RUN cd backend && npm install

# Copy backend and built frontend
COPY backend/ ./backend/
COPY --from=frontend /app/frontend/build
./backend/public

EXPOSE 5000

CMD ["node", "backend/server.js"]
```

## Build app:

```
PS D:\Documents\Sem_5\Web_Theory\Project\User\app> docker build -t amnanoor124/my-node-app:latest .
[+] Building 14.8s (18/18) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 614B
=> [internal] load metadata for docker.io/library/node:18
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [frontend 1/6] FROM docker.io/library/node:18@sha256:867be01f97d45cb7d89a8ef0b328d23e8207412ebec4564441ed8cab8cc4ecd
=> => resolve docker.io/library/node:18@sha256:867be01f97d45cb7d89a8ef0b328d23e8207412ebec4564441ed8cab8cc4ecd
=> [internal] load build context
=> => transferring context: 4.17MB
=> CACHED [stage-1 2/7] WORKDIR /app
=> CACHED [stage-1 3/7] COPY backend/package*.json ./backend/
=> CACHED [stage-1 4/7] RUN cd backend && npm install
=> CACHED [stage-1 5/7] COPY backend/ ./backend/
=> CACHED [stage-1 6/7] COPY backend/.env ./backend/
=> CACHED [frontend 2/6] WORKDIR /app/frontend
=> CACHED [frontend 3/6] COPY frontend/package*.json ./
=> CACHED [frontend 4/6] RUN npm install
=> CACHED [frontend 5/6] COPY frontend/ ./
=> CACHED [frontend 6/6] RUN npm run build
=> CACHED [stage-1 7/7] COPY --from=frontend /app/frontend/build ./backend/public
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:0f413a02c16d84e56bb86d0062b0ef8796c9e848b068faa89595411557ec0fef
=> => exporting config sha256:2b1b9e6924d41e23fcd75a0f680ac38f3b084ba35ca19eec88f04975662d636a
```

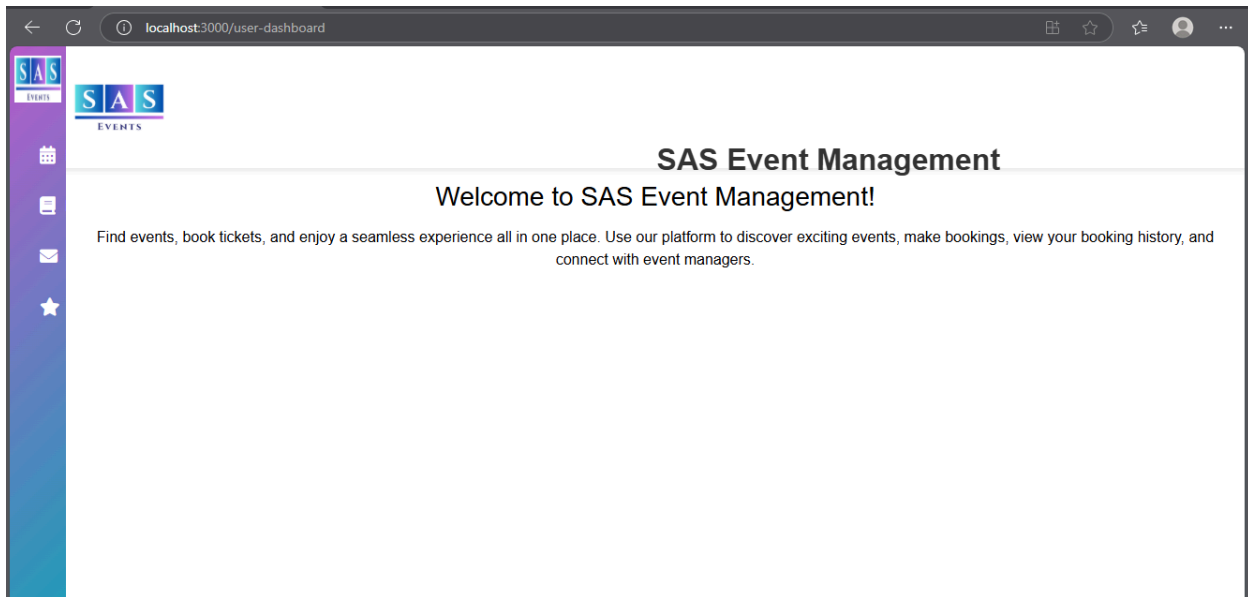
## Running app:

```

Run 'docker run --help' for more information
PS D:\Documents\Sem_5\Web_Theory\Project\User\app> docker run -p 5001:5000 -e MONGO_URI=mongodb+srv://i221529:2253149f@cluster0.pkql63w.mongodb.net/ amnanoor124/my-node-app
Server running on port 5000
MongoDB connected
Users updated with bookedEvents field.

```

On browser:



### Commands Involved:

- `Docker build -t amnanoor124/my-node-app:latest .`  
This command builds a Docker image from your Dockerfile and names it.
- `docker run -p 5001:5000 \`  
`-e MONGO_URI=mongodb+srv://i221529:2253149f@cluster0.pkql63w.mongodb.net/ \`  
`your-dockerhub-username/my-node-app`  
 Exposes port 5000 to the browser. Injects the MongoDB connection string as an environment variable. The image you want to run from Docker Hub

### 3. Push Code to GitHub

**git init** (initialize git in project folder)

**git add .** (add all files on staging area)

**git commit -m "Initial commit"** (commit files with message)

**git remote add origin**

[https://github.com/amna124/SCD\\_Event\\_Management.git](https://github.com/amna124/SCD_Event_Management.git) (add remote repository)

**git push -u origin main** (push files on remote repository)



**Included:**

Application code

Dockerfile

docker-compose.yml

README.md

Kubernetes files (deployment.yaml, service.yaml)

```

PS D:\Documents\Sem_5\Web_Theory\Project\User\app> git add .
warning: in the working copy of 'frontend/tailwind.config.js', LF will be replaced by CRLF the next time Git touches it
PS D:\Documents\Sem_5\Web_Theory\Project\User\app> git commit -m "Initial Commit with app and dockerfile"
[master (root-commit) 3976dd5] Initial Commit with app and dockerfile
80 files changed, 24701 insertions(+)
create mode 100644 .gitignore
create mode 100644 Dockerfile
create mode 100644 backend/controllers/eventController.js
create mode 100644 backend/controllers/sponsorController.js
create mode 100644 backend/controllers/userController.js
create mode 100644 backend/middleware/authMiddleware.js
create mode 100644 backend/middleware/isAuthenticated.js
create mode 100644 backend/middleware/sponsorMiddleware.js
create mode 100644 backend/models/Booking.js
create mode 100644 backend/models/Event.js
create mode 100644 backend/models/Feedback.js
create mode 100644 backend/models/Gallery.js
create mode 100644 backend/models/Sponsor.js
create mode 100644 backend/models/User.js
create mode 100644 backend/package-lock.json
create mode 100644 backend/package.json
create mode 100644 backend/routes/authRoutes.js
create mode 100644 backend/routes/bookingRoutes.js
create mode 100644 backend/routes/eventRoutes.js
create mode 100644 backend/routes/profileRoutes.js
create mode 100644 backend/routes/sponsorRoutes.js
create mode 100644 backend/routes/userRoutes.js
create mode 100644 backend/server.js
create mode 100644 backend/utils/generateToken.js
create mode 100644 frontend/.gitignore
create mode 100644 frontend/README.md
create mode 100644 frontend/package-lock.json
create mode 100644 frontend/package.json
create mode 100644 frontend/public/favicon.ico
create mode 100644 frontend/public/index.html

```

```

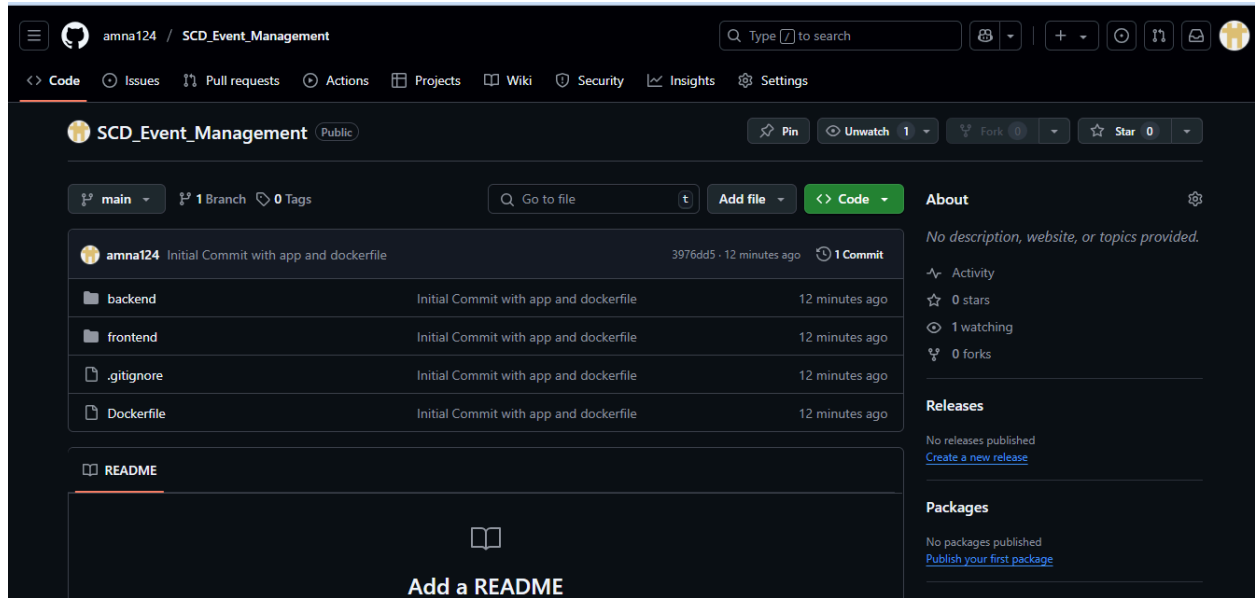
create mode 100644 frontend/tailwind.config.js
PS D:\Documents\Sem_5\Web_Theory\Project\User\app> git remote add origin https://github.com/amna124/SCD_Event_Management.git
PS D:\Documents\Sem_5\Web_Theory\Project\User\app> git push -u origin main

```

```

PS D:\Documents\Sem_5\Web_Theory\Project\User\app> git branch -M main
PS D:\Documents\Sem_5\Web_Theory\Project\User\app> git push -u origin main
Enumerating objects: 93, done.
Counting objects: 100% (93/93), done.
Delta compression using up to 4 threads
Compressing objects: 100% (91/91), done.
Writing objects: 100% (93/93), 971.23 KiB | 3.15 MiB/s, done.
Total 93 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/amna124/SCD_Event_Management.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS D:\Documents\Sem_5\Web_Theory\Project\User\app>

```



#### 4. Create Kubernetes Files:

- Create deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
```

```

        image:
amnanoor124/my-node-app:latest #  Use your
Docker Hub image name
        ports:
            - containerPort: 5000
        env:
            - name: MONGO_URI
              value:
"mongodb+srv://i221529:2253149$@cluster0.pkql6
3w.mongodb.net/"

```

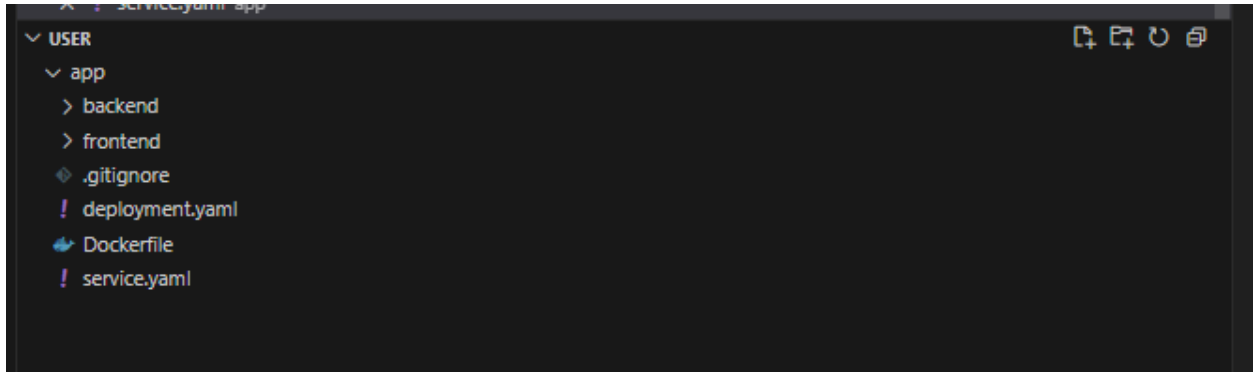
- **Create service.yaml**

```

apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  type: NodePort
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30007

```

**Project Structure:**



## 5. Start the Kubernetes Cluster:

```
PS C:\WINDOWS\system32> D:
PS D:\> cd "D:\Documents\Sem_5\Web_Thoery\Project\User\app"
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> minikube start --driver=docker
* minikube v1.35.0 on Microsoft Windows 10 Pro 10.0.19045.5737 Build 19045.5737
```

**minikube start:** – Launches a **local single-node Kubernetes cluster** using Minikube.

**minikube status:** Shows the **current status** of the Minikube cluster and its components (e.g., host, kubelet, apiserver).

```

Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

Warning: PowerShell detected that you might be using a screen reader and has disabled PSReadLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSReadLine'.

PS C:\WINDOWS\system32> D:
PS D:\> cd "D:\Documents\Sem_5\Web_Thoery\Project\User\app"
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> minikube start --driver=docker
* minikube v1.35.0 on Microsoft Windows 10 Pro 10.0.19045.5737 Build 19045.5737
* Using the docker driver based on user configuration
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.46 ...
* Downloading Kubernetes v1.32.0 preload ...
  > preloaded-images-k8s-v18-v1...: 333.57 MiB / 333.57 MiB 100.00% 241.90
  > gcr.io/k8s-minikube/kicbase...: 500.31 MiB / 500.31 MiB 100.00% 200.68
* Creating docker container (CPUs=2, Memory=1965MB) ...
! Failing to connect to https://registry.k8s.io/ from inside the minikube container
* To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
* Preparing Kubernetes v1.32.0 on Docker 27.4.1 ...
  - Generating certificates and keys ...
  - Booting up control plane ...
  - Configuring RBAC rules ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: default-storageclass, storage-provisioner
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> minikube start --driver=docker

* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> minikube docker-env
$Env:DOCKER_TLS_VERIFY = "1"
$Env:DOCKER_HOST = "tcp://127.0.0.1:49214"
$Env:DOCKER_CERT_PATH = "C:\Users\Pak\.minikube\certs"
$Env:MINIKUBE_ACTIVE_DOCKERD = "minikube"
# To point your shell to minikube's docker-daemon, run:
# & minikube -p minikube docker-env --shell powershell | Invoke-Expression
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app>

```

## eval \$(minikube docker-env)

This command configures your terminal to use Minikube's internal Docker daemon, allowing you to build Docker images directly inside the Minikube VM.

```

# & minikube -p minikube docker-env --shell powershell | Invoke-Expression
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> eval $(minikube -p minikube docker-env)
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> & minikube -p minikube docker-env | Invoke-Expression
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app>

```

Again building to make files up-to date:

```

=> CACHED [final 4/8] WORKDIR /app/backend 0.0s
[+] Building 752.4s (19/19) FINISHED docker:default
=> [internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 973B 0.1s
=> [internal] load metadata for docker.io/library/node:18 2.4s
=> [auth] library/node:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.1s
=> => transferring context: 139B 0.0s
=> [frontend 1/6] FROM docker.io/library/node:18@sha256:867be01f97d45cb7d89a8ef0b328d23e8207412ebec4564441ed8cab 0.0s
=> [internal] load build context 21.5s
=> => transferring context: 4.17MB 21.3s
=> CACHED [frontend 2/6] WORKDIR /app/frontend 0.0s
=> CACHED [frontend 3/6] COPY frontend/package*.json ./ 0.0s
=> [frontend 4/6] RUN npm install 236.4s
=> CACHED [final 2/8] WORKDIR /app 0.0s
=> CACHED [final 3/8] COPY backend/package*.json ./backend/ 0.0s
=> CACHED [final 4/8] WORKDIR /app/backend 0.0s
=> CACHED [final 5/8] RUN npm config set registry https://registry.npmjs.org/ && npm config set fetch-retrie 0.0s
=> CACHED [final 6/8] WORKDIR /app 0.0s
=> CACHED [final 7/8] COPY backend/ ./backend/ 0.0s
=> [frontend 5/6] COPY frontend/ ./ 264.9s
=> [frontend 6/6] RUN npm run build 155.7s
=> [final 8/8] COPY --from=frontend /app/frontend/build ./backend/public 3.0s
=> exporting to image 34.1s
=> => exporting layers 33.8s
=> => writing image sha256:08789043d10e1241b26f7524e91ca9943cfd1db1dfd32974adac9d9d95ef2d34d 0.0s
=> => naming to docker.io/amnanoor124/my-node-app:latest 0.0s
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app>

```

## 6. Setup Docker Hub

**docker build -t amnanoor124/my-node-app:latest . -**

Builds a Docker image from the current directory (.) and tags it as amnanoor124/my-node-app:latest for Docker Hub.

**docker push amnanoor124/my-node-app:latest** – Uploads (pushes) the tagged Docker image to the Docker Hub repository amnanoor124/my-node-app.

```

PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> docker login
Authenticating with existing credentials... [Username: amnanoor124]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> docker tag my-node-app:latest amnanoor124/my-node-app:latest
Error response from daemon: No such image: my-node-app:latest
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> docker push amnanoor124/my-node-app:latest
The push refers to repository [docker.io/amnanoor124/my-node-app]
a3fdbba48e4c8: Pushing [=====] 3.012MB
11cdecccd4ee: Pushing [=====] 8.479MB/24.84MB
5f70bf18a086: Pushed
39ff463c4382: Pushing [=====] 8.533MB/31.71MB
ab29f664e45c: Pushed
68022cabb018: Pushing 1.536kB
4ef4dcd20f76: Waiting
438e733c9a0c: Waiting
f7bcf5aff5f1: Waiting
2f3ebdcf6a27: Waiting
bf9c09fb6f3a: Waiting
fcb8c0ae5d6: Waiting
8ce3e08e661a: Waiting
247fff7b7158d: Waiting

```

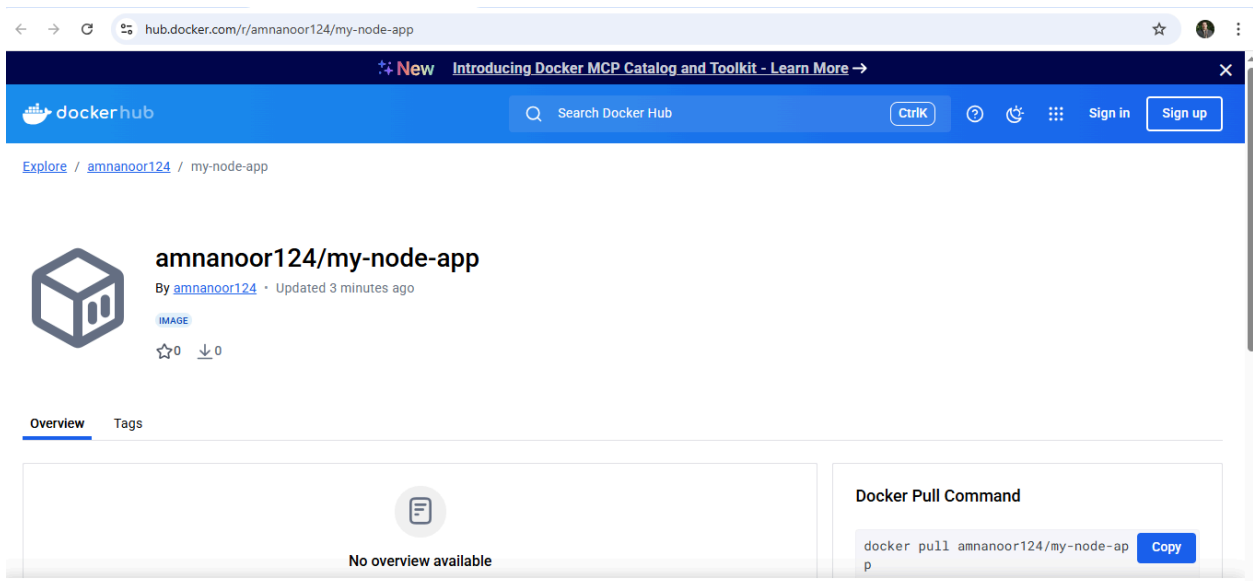
```

PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> docker push amnanoor124/my-node-app:latest
The push refers to repository [docker.io/amnanoor124/my-node-app]
a3fdbba48e4c8: Pushed
11cdeecdd4ee: Pushed
5f70bf18a086: Pushed
39ff463c4382: Pushed
ab29f664e45c: Pushed
68022cabb018: Pushed
4ef4dcd20f76: Mounted from library/node
438e733c9a0c: Mounted from library/node
f7bcf5aff5f1: Mounted from library/node
2f3ebdcf6a27: Mounted from library/node
bf9c09fb6f3a: Mounted from library/node
fcb8c0ae5d6: Mounted from library/node
8ce3e08e661a: Mounted from library/node
247fffb7158d: Mounted from library/node
latest: digest: sha256:67c4aa34c022c723c48cabe383630428d9f9028c46c540976ddd4cdacbd732d4 size: 3465
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app>

```

Link for published image:

<https://hub.docker.com/r/amnanoor124/my-node-app>



## Set Up GitHub Actions Workflow

- Self-Hosted GitHub Runner:

github.com/amna124/SCD\_Event\_Management/settings/actions/runners

amna124 / SCD\_Event\_Management

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

General

Runners

Webhooks

Environments

Codespaces

## Runners

New self-hosted runner

Host your own runners and customize the environment used to run jobs in your GitHub Actions workflows. [Learn more about self-hosted runners.](#)

There are no runners configured

[Learn more about using runners](#) to run actions on your own servers.

```
latest: digest: sha256:67c4aa34c022c723c48cabe383630428d9f9028c46c540976ddd4cdacbd732d4 size: 3465
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> mkdir \actions-runner

Directory: D:\

Mode                LastWriteTime         Length Name
----                -
d-----          5/11/2025   6:31 PM              actions-runner

PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> cd \actions-runner
PS D:\actions-runner> Invoke-WebRequest -Uri https://github.com/actions/runner/releases/download/v2.323.0/actions-runner-win-x64-2.323.0.zip -OutFile actions-runner-win-x64-2.323.0.zip
PS D:\actions-runner> Add-Type -AssemblyName System.IO.Compression.FileSystem ; [System.IO.Compression.ZipFile]::ExtractToDirectory("$PWD/actions-runner-win-x64-2.323.0.zip", "$PWD")
PS D:\actions-runner>
```



```
PWD/actions-runner-win-x64-2.323.0.zip", "$PWD")
PS D:\actions-runner> ./config.cmd --url https://github.com/amna124/SCD_Event_Management --token A67BSFZAJA42NLKTAK40X2TIEC2BI

-----
GitHub Actions
Self-hosted runner registration
-----

# Authentication

✓ Connected to GitHub

# Runner Registration

Enter the name of the runner group to add this runner to: [press Enter for Default]

Enter the name of runner: [press Enter for DESKTOP-NKS73I4]

This runner will have the following labels: 'self-hosted', 'Windows', 'X64'
Enter any additional labels (ex. label-1,label-2): [press Enter to skip]

✓ Runner successfully added
✓ Runner connection is good

# Runner settings

Enter name of work folder: [press Enter for _work]

✓ Settings Saved.

Would you like to run the runner as service? (Y/N) [press Enter for N]
PS D:\actions-runner>
```

Running Successfully:

To run self-hosted runner: `./run.cmd`

```
Would you like to run the runner as service? (Y/N) [press Enter for N]
PS D:\actions-runner> ./run.cmd
1 file(s) copied.

✓ Connected to GitHub

Current runner version: '2.323.0'
2025-05-11 13:41:53Z: Listening for Jobs
```

- Create Deploy.yml file:

```
name: Build and Deploy to Minikube

on:
  push:
    branches:
      - main

jobs:
```

```

build-deploy:
  runs-on: self-hosted

  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Configure Docker to use Minikube's
      Docker daemon
      shell: powershell
      run: |
        Write-Host "Setting Docker to use
Minikube's Docker daemon..."
        & minikube -p minikube docker-env |
Invoke-Expression

    - name: Build Docker image
      run: |
        docker build -t
amnanoor124/my-node-app:latest .

    - name: Login to Docker Hub
      run: echo "${{ secrets.DOCKER_PASSWORD }}"
| docker login -u "${{ secrets.DOCKER_USERNAME }}"
--password-stdin

    - name: Push Docker image to Docker Hub
      run: docker push
amnanoor124/my-node-app:latest

```

```
- name: Deploy to Minikube
  run: |
    cd app
    kubectl apply -f deployment.yaml
    kubectl apply -f service.yaml
```

The screenshot shows the GitHub Actions settings page for the repository 'SCD\_Event\_Management'. The 'Runners' section is active, displaying a table with one runner: 'DESKTOP-NKS7314', which is self-hosted, on Windows, X64, and currently idle. A 'New self-hosted runner' button is visible. Below this, the 'Repository secrets' section is shown, indicating that the environment has no secrets. A 'Manage environment secrets' button is present. The 'Repository secrets' table lists two secrets: 'DOCKER\_PASSWORD' (updated 'now') and 'DOCKER\_USERNAME' (updated '10 minutes ago'). Both have edit and delete icons. The left sidebar shows the navigation menu with 'Runners' and 'Repository secrets' highlighted.

## 7. Trigger Local Deployment

Making sure that we're inside minikube: a local single-node Kubernetes cluster. Unless you switch context, **kubectl** is now automatically connected to Minikube.

```
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> kubectl config current-context
minikube
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app>
```

```
error: the path "k8s/deployment.yaml" does not exist
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> kubectl apply -f deployment.yaml
deployment.apps/my-app created
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> kubectl apply -f service.yaml
service/my-app-service created
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app>
```

```
my-app-595448c6fc-plpwz 1/1 Running 0 103m
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> kubectl get all -n event-management
```

| NAME                        | READY | STATUS  | RESTARTS | AGE  |
|-----------------------------|-------|---------|----------|------|
| pod/my-app-595448c6fc-m81v2 | 1/1   | Running | 0        | 103m |
| pod/my-app-595448c6fc-plpwz | 1/1   | Running | 0        | 103m |

| NAME                   | TYPE     | CLUSTER-IP     | EXTERNAL-IP | PORT(S)      | AGE  |
|------------------------|----------|----------------|-------------|--------------|------|
| service/my-app-service | NodePort | 10.100.105.122 | <none>      | 80:30007/TCP | 102m |

| NAME                   | READY | UP-TO-DATE | AVAILABLE | AGE  |
|------------------------|-------|------------|-----------|------|
| deployment.apps/my-app | 2/2   | 2          | 2         | 103m |

| NAME                              | DESIRED | CURRENT | READY | AGE  |
|-----------------------------------|---------|---------|-------|------|
| replicaset.apps/my-app-595448c6fc | 2       | 2       | 2     | 103m |

```
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app>
```

`kubectl apply -f deployment.yaml` – Applies the Kubernetes deployment configuration to create/update pods and replica sets.

`kubectl apply -f service.yaml` – Applies the service configuration to expose the deployed application within the cluster.

`kubectl get pods -o wide` – Lists all running pods with detailed information like IP, node, and status.

`kubectl get svc -o wide` – Displays all services in the cluster with extended details like cluster IP and port mappings.

`kubectl get nodes` – Shows all nodes in the Kubernetes cluster along with their status and roles.

`minikube service my-app-service` – Opens the deployed app in your default browser by routing the NodePort service through Minikube.

Run successful:

```

replicaset.apps/my-app-595448c6fc 2 2 2 103m
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> minikube service my-app-service -n event-management

```

| NAMESPACE        | NAME           | TARGET PORT | URL                       |
|------------------|----------------|-------------|---------------------------|
| event-management | my-app-service | 80          | http://192.168.49.2:30007 |

```

* Starting tunnel for service my-app-service.

```

| NAMESPACE        | NAME           | TARGET PORT | URL                    |
|------------------|----------------|-------------|------------------------|
| event-management | my-app-service |             | http://127.0.0.1:54010 |

```

* Opening service event-management/my-app-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.

```

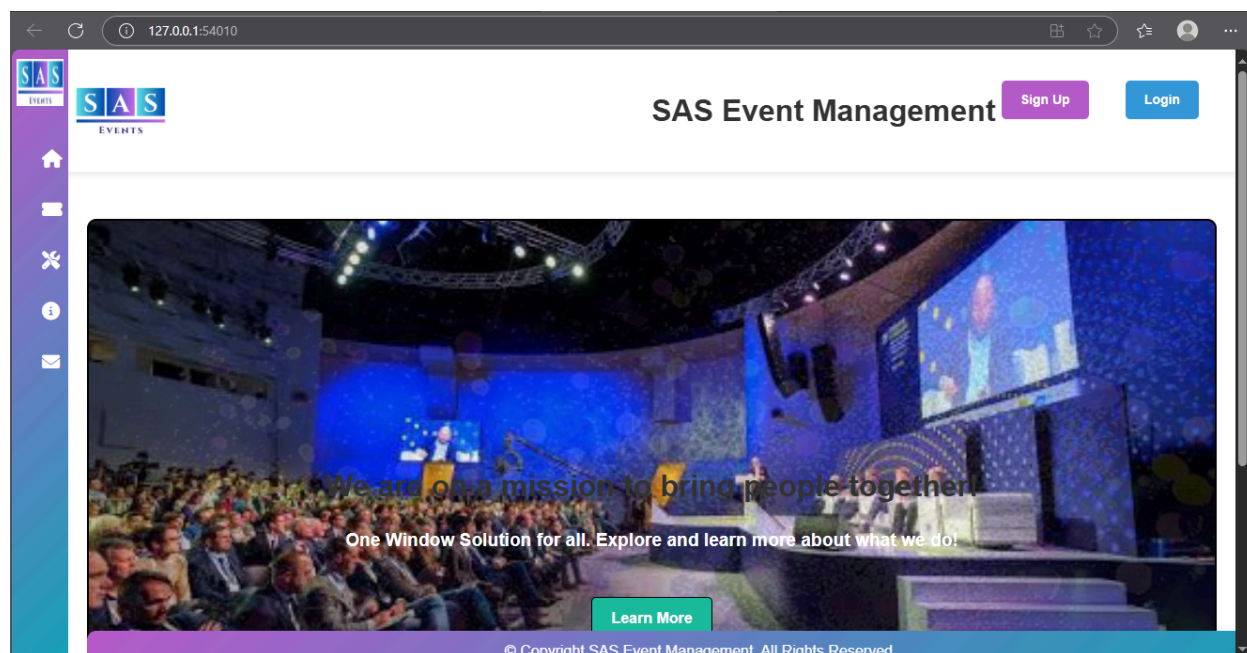
Response from backend:

```

PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> kubectl logs -n event-management -l app=my-app
Server running on port 5000
MongoDB connected
Users updated with bookedEvents field.
Server running on port 5000
MongoDB connected
Users updated with bookedEvents field.
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app>

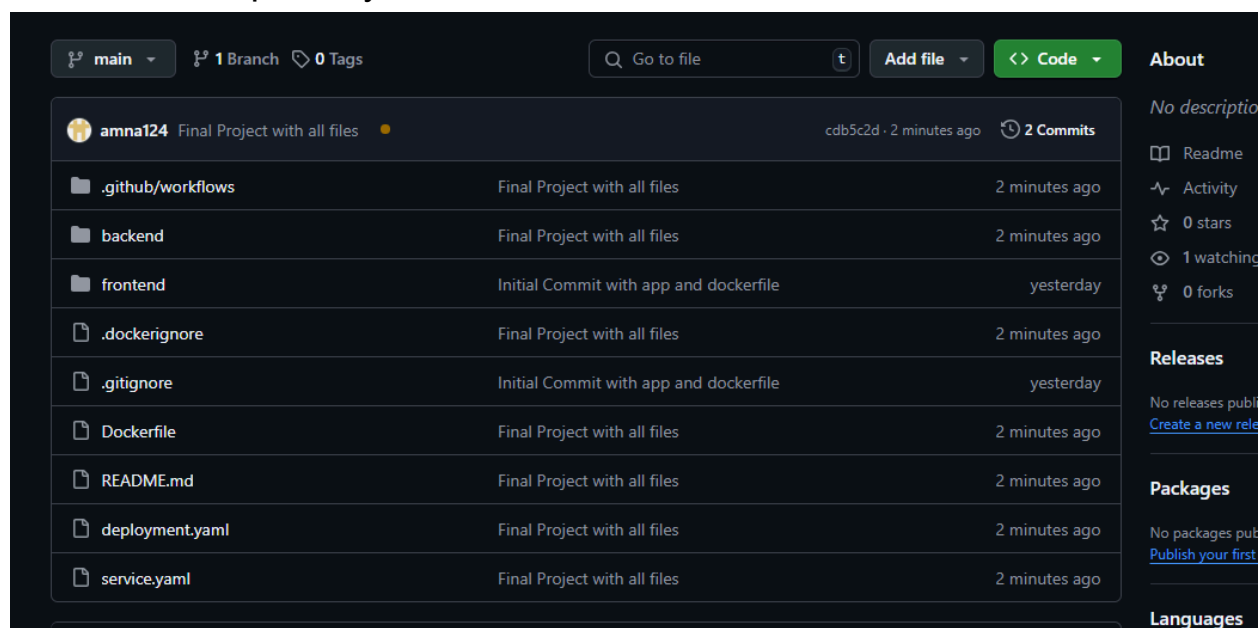
```

Response from frontend



```
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> kubectl get pods -n event-management
NAME                                READY   STATUS    RESTARTS   AGE
my-app-5b7b7d98c-5wrnd             1/1     Running   1 (13m ago)  67m
my-app-5b7b7d98c-65mpj             1/1     Running   1 (13m ago)  68m
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> kubectl get svc -n event-management
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
my-app-service      NodePort    10.100.105.122  <none>        80:30007/TCP     4h34m
```

Final Github repository:



Issues:

1. In the beginning, the build command was taking too long and failed at last because node\_modules were large and dockerfile was running it for both backend and frontend.

```
[+] Building 261.7s (10/12)
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 638B
=> [internal] load metadata for docker.io/library/node:18
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/8] FROM docker.io/library/node:18@sha256:867be01f97d45cb7d89a8ef0b328d23e8207412ebec4564441ed8cabc8cc4e
=> => resolve docker.io/library/node:18@sha256:867be01f97d45cb7d89a8ef0b328d23e8207412ebec4564441ed8cabc8cc4e
=> [internal] load build context
=> => transferring context: 15.87MB
=> CACHED [2/8] WORKDIR /app
=> [3/8] COPY backend/package*.json ./backend/
=> [4/8] RUN cd backend && npm install
=> [5/8] COPY frontend/package*.json ./frontend/
=> ERROR [6/8] RUN cd frontend && npm install && npm run build
-----
> [6/8] RUN cd frontend && npm install && npm run build:
4.924 npm warn EBADENGINE Unsupported engine {
4.924 npm warn EBADENGINE   package: 'react-router@7.6.0',
4.924 npm warn EBADENGINE   required: { node: '>=20.0.0' },
4.924 npm warn EBADENGINE   current: { node: 'v18.20.8', npm: '10.8.2' }
4.924 npm warn EBADENGINE }
4.928 npm warn EBADENGINE Unsupported engine {
```

### Solution:

We created .dockerignore file and added extra files in it. Below is my file:

```
> backend
> frontend
. .dockerignore
. Dockerfile
```

```
node_modules
frontend/node_modules
backend/node_modules
.git
.gitignore
Dockerfile
build
dist
.env
```

2. There was routing path error when after build, we tried to run the container:

```
Error: Cannot find module './Routes/userRoutes'
```

### Solution:

We fixed the routing path from ./Routes/userRoutes to ./routes/userRoutes and again build the app and run the container.

3. There was some .env file problem because inside Docker, process.env.MONGO\_URI is undefined, because .env is not being passed or copied into the container correctly.

```
MongooseError: The `uri` parameter to `openUri()` must be a string, got "undefined".
```

### Solution:

While running We used. This manually sets the env variable at runtime.:

```
docker run -p 5000:5000 \  
-e
```

```
MONGO_URI=mongodb+srv://i221529:2253149$@cluster0.pkql6  
3w.mongodb.net/ \  
amnanoor124/my-node-app
```

4. While pushing code on git, we faced this problem because we didn't created main branch and switched to it:

```
error: src refspec main does not match any
```

### Solution



```
git branch -M main      # Renames our current branch to 'main'
git push -u origin main
```

5. At very last step, our frontend was running but backend was not responding in minikube:

```
Error updating users: MongooseError: Operation `users.find()` buffering timed out after 10000ms
```

### Solution:

#### MongoDB Atlas → Network Access

Made sure **our Minikube IP** (e.g., 192.168.49.2) is in the list

At last, we got response from mongodb:

```
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app> kubectl logs -n event-management -l app=my-app
Server running on port 5000
MongoDB connected
Users updated with bookedEvents field.
Server running on port 5000
MongoDB connected
Users updated with bookedEvents field.
PS D:\Documents\Sem_5\Web_Thoery\Project\User\app>
```

### Project running instructions:

#### 1. Prerequisites

Ensure the following are installed on system:

- Node.js
- Docker Desktop
- Minikube
- kubectl (Kubernetes CLI)
- A GitHub account
- A Docker Hub account

## 2. Clone the Repository

- `git clone https://github.com/amna124/SCD_Event_Management.git`
- `cd SCD_Event_Management`
- `git clone https://github.com/amna124/SCD_Event_Management.git`
- `cd project`
- `Docker build -t amnanoor124/my-node-app:latest .`

## 3. Set Environment Variables

Create a `.env` file in both backend/.

## 4. Build and Push Docker Image

Make sure `Dockerfile` is in the root of the project and then run:

```
docker build -t amnanoor124/my-node-app:latest .
```

```
docker push amnanoor124/my-node-app:latest
```

## 5. Start Minikube

```
minikube start
```

- Ensure using local Docker now:

```
minikube docker-env --unset
```

```
minikube start
```

```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

```
minikube service my-app-service
```

## 6. Create Kubernetes Namespace

```
kubectl create namespace event-management
```

## 7. Deploy App to Minikube

Make sure your Kubernetes files are present:

- `deployment.yaml`
- `service.yaml`

Then run:

```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

```
kubectl rollout restart deployment my-app -n event-management
```

## 8. Access the Application

To open the app in your browser:

```
minikube service my-app-service -n event-management
```

## 9. CI/CD with GitHub Actions

A self-hosted GitHub Actions runner is configured to:

- Automatically build and push Docker image
- Redeploy to Minikube on each push to main branch

The workflow is defined in:

```
.github/workflows/deploy.yml
```

## Operating System Selection Requirement

**Operating System / Setup Chosen:**

For this project, I used **Windows 10 pro version 22** with the following local development tools:

- **Docker Desktop for Windows**
  - **Minikube with Docker driver**
  - **GitHub Actions (self-hosted runner on Windows)**
  - **VS Code** as the primary development editor
- 

#### **Why I Selected This Setup:**

- **Windows 10 pro version22** was my primary OS and readily available on my machine.
  - **Docker Desktop** provides a user-friendly and integrated experience for building and running containers on Windows.
  - **Minikube** allowed me to run a full Kubernetes cluster locally without needing cloud access, which was essential for testing deployment scripts.
  - Using a **self-hosted GitHub Actions runner on the same machine** allowed seamless CI/CD integration with my Minikube cluster and Docker daemon.
- 

#### **Advantages Encountered:**

- **Local-only, cloud-free Kubernetes setup:** Using Minikube avoided the need for cloud accounts or billing and was fast to reset or

redeploy.

- **End-to-end control:** Running Docker, Kubernetes, and GitHub Actions locally gave me full visibility into all layers of the stack.
  - **Strong IDE support:** VS Code with Docker and Kubernetes extensions made development and YAML editing easier and faster.
  - **Environment parity:** Docker ensured consistency between local and production builds.
- 

### Challenges Faced:

- **Docker memory limits:** On Windows, Docker occasionally ran out of memory during image builds (~500MB context size), requiring cleanup or `.dockerignore` optimization.
  - **Network connectivity with MongoDB Atlas:** Minikube's internal IP was not whitelisted by default; I had to manually add it to MongoDB Atlas.
  - **Long build times:** Initial Docker builds (especially without `.dockerignore`) were slow. I solved this by minimizing the build context and using multi-stage Docker builds.
  - **Self-hosted runner management:** Keeping the runner active required a dedicated terminal session, and error tracing had to be done manually through GitHub's Actions logs.
- 

### How It Affected My Workflow:

- **Performance:** Once properly configured, the local Minikube cluster ran smoothly. Pod startup and redeployment were fast.
  - **Tool Compatibility:** Docker, kubectl, and GitHub Actions worked reliably together after setup. All core tooling was compatible with my Windows environment.
  - **Learning Opportunity:** Managing the full toolchain locally gave me a strong understanding of how Docker, Kubernetes, and CI/CD systems interact.
-